



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
INGENIERÍA DE SOFTWARE II



INFORME INTEGRAL PRODUCTO I:

“FUNDAMENTOS DE LA GESTIÓN DE PROYECTOS Y PRODUCTOS DE SOFTWARE”

Docente:

Félix Marquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Ciudad Guayana, Septiembre del 2026

ÍNDICE

Descripción	página
Introducción	1
Parte I: Marco Teórico y Fundamentos Disciplinarios	2
1. De la Crisis del Software a la Complejidad Cynefin, Ágil y DevOps.....	2
2. Las 4 Dimensiones de Pressman y el Costo del Cambio de Boehm.....	2
3. Gestión de Proyectos vs. Gestión de Productos (Entrega de Valor).....	3
Parte II: Resolución de los Ejercicios Prácticos de Reflexión	3
Ejercicio 1: Traducción de Necesidades a Requerimientos y Actores (Caso Hospital Público).....	3
Ejercicio 2: Análisis de las 4 Dimensiones de Pressman y Errores Clásicos (Caso Startup Delivery).....	5
Ejercicio 3: PMBOK 6 (Procesos) vs. PMBOK 7 (Principios) en la Gestión de Requisitos.....	6
Ejercicio 4: Discusión Crítica: "El mayor riesgo es que el software se construya perfectamente pero nadie lo use"	8
Conclusiones	9
Referencias	10

Introducción

La ingeniería de software ha experimentado una profunda transformación conceptual y metodológica desde su identificación formal en la Conferencia de la OTAN en Garmisch (1968). Durante décadas, el sector tecnológico estuvo condicionado por la denominada "crisis del software", un estado recurrente en el que los proyectos fracasaban no por deficiencias intrínsecas en la capacidad de codificación, sino por una gestión inadecuada de requerimientos, barreras de comunicación multidireccional y liderazgos basados en el control burocrático de modelos secuenciales rígidos.

El desarrollo moderno de software opera en dominios complejos y dinámicos gobernados por el marco Cynefin de Snowden (2002), donde el valor ya no radica en la producción masiva de entregables documentales (*outputs*) dentro del rígido triángulo de hierro, sino en la entrega continua, iterativa y medible de valor real de negocio (*outcomes*). Esta evolución consolidó el paso de la "Gestión de Proyectos" hacia la "Gestión de Productos", apalancada en la cultura DevOps, la automatización del ciclo de vida y el liderazgo servicial (Greenleaf, 1977).

El presente informe técnico tiene por objetivo estructurar el marco teórico fundamental de la Unidad 1 y resolver de forma analítica los cuatro ejercicios de reflexión planteados en la guía pedagógica oficial. La discusión se sustenta rigurosamente en los estándares internacionales PMBOK 7, SWEBOK V4, ISO/IEC/IEEE 29148, ISO/IEC 20926 e ISO 31000:2018, integrando además la comparativa de actualización hacia el PMBOK 8 y su impacto en la ingeniería contemporánea.

Parte I: Marco Teórico y Fundamentos Disciplinarios

1. De la Crisis del Software a la Complejidad Cynefin, Ágil y DevOps

La crisis del software evidenció la inviabilidad de aplicar modelos secuenciales predictivos (Cascada y Modelo en V) al desarrollo tecnológico. Mientras el hardware crecía exponencialmente bajo la Ley de Moore, los proyectos acumulaban desviaciones presupuestarias superiores al 180% y tasas de fracaso documentadas por el Standish Group (CHAOS Report) superiores al 70%. Hoy en día, la crisis ha mutado hacia la gestión de la Complejidad Esencial: según el modelo Cynefin (Snowden, 2002), el software pertenece a sistemas complejos donde los resultados solo se comprenden mediante experimentación empírica y bucles rápidos de inspección y adaptación.

El Manifiesto Ágil (2001) propuso una ruptura filosófica basada en 4 valores cardinales (individuos e interacciones, software funcionando, colaboración y respuesta ante el cambio). Posteriormente, DevOps y Site Reliability Engineering (SRE) eliminaron el "muro de la confusión" entre desarrolladores y operadores, habilitando pipelines de integración y entrega continua (CI/CD) que reducen el tiempo de ciclo (*lead time*) de meses a minutos con alta disponibilidad.

2. Las 4 Dimensiones de Pressman y el Costo del Cambio de Boehm

Pressman (2014) define el ecosistema de software a través de cuatro dimensiones interdependientes que deben operar en equilibrio armónico:

- **Personas:** El capital humano, el equipo técnico, usuarios y stakeholders. El liderazgo servicial (Greenleaf, 1977) es el motor que coordina el talento.
- **Proceso:** El marco metodológico que articula personas y tecnología (Scrum, Kanban, DevOps).
- **Producto:** La solución de valor que resuelve los problemas de negocio y satisface al usuario.
- **Tecnología:** El conjunto de herramientas, lenguajes, frameworks e infraestructuras utilizadas.

La Curva de Boehm (1981) fundamenta la importancia crítica de la ingeniería de requisitos al demostrar que corregir un defecto durante la fase de mantenimiento o producción cuesta hasta 100 veces más que subsanar durante la etapa inicial de licitación. En consecuencia, una mala gestión de requisitos y de interesados genera retrabajo, desperdicio y deuda técnica acumulativa.

3. Gestión de Proyectos vs. Gestión de Productos (Entrega de Valor)

Criterio	Gestión de Proyectos (Project-Driven)	Gestión de Productos (Product-Driven)
Foco Central	Entregar un alcance cerrado (<i>Output</i>) dentro del tiempo y presupuesto previstos.	Maximizar el impacto y la entrega continua de valor sostenible (<i>Outcome</i>).
Temporalidad	Finita, con fechas estrictas de inicio, cierre y desmantelamiento de equipo.	Continua a lo largo de todo el ciclo de vida del producto en el mercado.
Métricas de Éxito	Conformidad con la línea base (Varianza de Costo y Cronograma).	Retención, adopción, ROI, Costo de Adquisición (CAC) y Lifetime Value (LTV).

Parte II: Resolución de los Ejercicios Prácticos de Reflexión

Ejercicio 1: Traducción de Necesidades a Requerimientos y Actores (Caso Hospital Público)

Contexto: Reducción del tiempo de espera de pacientes en el área de urgencias de un hospital público.

Marco Aplicado: ISO/IEC/IEEE 29148, SWEBOK V4 y Principio de Interesados del PMBOK7.

Actor Clave	Necesidad Subyacente (Problema)	Requerimiento Específico (SRS)	Tipo
Paciente / Familiar	Incertidumbre y ansiedad por desinformación sobre su turno de atención.	RF-01: El sistema debe generar un identificador QR interactivo con alertas SMS que actualicen en tiempo real el tiempo estimado de espera.	Funcional
Enfermero de Triage	Riesgo de error humano y lentitud al categorizar manualmente la gravedad del paciente.	RF-02: El sistema debe sugerir automáticamente el nivel de prioridad (1 al 5) según el protocolo Manchester al registrar los signos vitales.	Funcional
Director Médico	Falta de visibilidad sobre cuellos de botella y sobrecarga de personal asistencial.	RF-03: El sistema debe emitir alertas telemétricas cuando el promedio de espera en triaje supere el umbral máximo de 20 minutos.	Funcional
Infraestructura Hospitalaria	Riesgo de colapso de terminales ante picos de demanda masiva o fallas de red.	RNF-01: La latencia de sincronización entre estaciones de triaje no debe superar los 500 ms bajo 500 sesiones concurrentes (Disponibilidad 99.99%).	No Funcional

Reflexión sobre Validación Temprana con Prototipado e IA: El prototipado interactivo (Figma) permite validar flujos ergonómicos directamente con médicos y enfermeros antes de codificar, neutralizando el impacto de la curva de Boehm. Paralelamente, la IA Generativa permite analizar volúmenes históricos de ingresos para detectar patrones de saturación clínica y sugerir requerimientos no funcionales de ciberseguridad médica (HIPAA) y resiliencia operativa.

Ejercicio 2: Análisis de las 4 Dimensiones de Pressman y Errores Clásicos (Caso Startup Delivery)

Contexto: Startup de delivery donde el director impone Kubernetes/microservicios a un equipo de desarrollo que solo domina PHP monolítico, sin proceso formal de cambios y admitiendo pagos con criptomonedas a mitad del proyecto sin presupuesto adicional.

1. Desbalances en las 4 Dimensiones de Pressman:

- **Personas vs. Tecnología:** Sobreingeniería severa al imponer infraestructura distribuida (Kubernetes) sobre un equipo capacitado exclusivamente en PHP monolítico, generando sobrecarga cognitiva y parálisis técnica.
- **Proceso:** Nula disciplina metodológica y ausencia total de un proceso formal para evaluar y canalizar solicitudes de cambio.
- **Producto / Alcance:** Alteración no planificada del alcance funcional (criptomonedas) que desestabiliza el MVP y destruye la planificación presupuestaria.

2. Errores Clásicos (Brooks / Standish Group):

- **Corrupción del Alcance (Scope Creep):** Integración de funcionalidades críticas sin control de varianza presupuestaria ni temporal.
- **Síndrome de la "Bala de Plata":** Creencia errónea de que adoptar microservicios en Kubernetes solventará las fallas estructurales del desarrollo.
- **Optimismo Injustificado (Ley de Hofstadter / Brooks):** Subestimar la prolongada curva de aprendizaje de arquitecturas complejas en equipos sin experiencia previa.

3. Solución de Mitigación Inmediata del Líder Servicial:

- Reorientar la arquitectura técnica hacia un Monolito Modular en PHP (Laravel) contenido en Docker básico, capitalizando las fortalezas reales del equipo para lanzar el MVP con rapidez.
- Negociar con el cliente la postergación de la pasarela de criptomonedas al Product Backlog de una versión 2.0 posterior, priorizando métodos de pago tradicionales para validar el modelo de negocio.
- Implementar un marco de flujo liviano (Kanban) con límites de Trabajo en Progreso (WIP) y reglas explícitas de gestión de cambios.

Además de los errores identificados, se observa una falta de participación del usuario final en el proceso de definición del producto. Los repartidores y comercios afiliados no fueron consultados sobre sus necesidades reales, lo que aumenta el riesgo de que el software resuelva problemas imaginarios. Desde el rol de Líder Servicial, convocaría a una sesión de design thinking con repartidores y dueños de locales para rediseñar el MVP priorizando sus puntos de dolor reales, antes de seguir invirtiendo en funcionalidades especulativas.

Ejercicio 3: PMBOK 6 (Procesos) vs. PMBOK 7 (Principios) en la Gestión de Requisitos

Mientras que el PMBOK 6 concebía la gestión de requerimientos a través de 49 procesos secuenciales cerrados y matrices rígidas de trazabilidad con apego estricto al contrato, el PMBOK 7 promueve la entrega holística de valor, la adaptabilidad (Tailoring), el pensamiento sistémico y la navegación activa en la incertidumbre.

Escenario Práctico	Gestión bajo PMBOK 6 (Enfoque en Procesos)	Gestión bajo PMBOK 7 (Enfoque en Principios de Valor)
Ejemplo 1: Solicitud de panel telemétrico en tiempo real	El requerimiento se rechaza inicialmente por no estar en la EDT/WBS base y se envía al Comité de Control Integrado de Cambios (proceso 4.6), retrasando la entrega por semanas de trámites burocráticos.	Bajo los principios de "Enfoque en el Valor" y "Adaptabilidad", el equipo evalúa el beneficio real de negocio y refactoriza colaborativamente el Product Backlog junto al Product Owner de inmediato.
Ejemplo 2: Detección de Deuda Técnica en Autenticación	Se posterga la refactorización para evitar que se degrade el Índice de Desempeño del Cronograma (SPI), acumulando vulnerabilidades de ciberseguridad.	Bajo los principios de "Administración Diligente (Stewardship)" y "Calidad Incorporada", se prioriza la remediación de la deuda técnica como un riesgo estructural indispensable.

Un aspecto clave que diferencia ambos enfoques es el rol del Product Owner. En PMBOK 6, el analista de requisitos documenta y el Comité de Cambios aprueba; en PMBOK 7, el Product Owner decide en colaboración con el equipo basándose en el valor de negocio. Esto reduce drásticamente los tiempos de respuesta ante cambios del mercado.

Además, el 'Tailoring' (adaptación) en PMBOK 7 permite que un proyecto de software misión crítica (ej. sistema de control de vuelos) use un enfoque más predictivo, mientras que un MVP de startup use uno adaptativo, todo dentro del mismo marco de principios. Esto es imposible en PMBOK 6, donde el proceso es único para todos los proyectos.

Ejercicio 4: Discusión Crítica: "El mayor riesgo es que el software se construya perfectamente pero nadie lo use"

La afirmación plantea un desafío epistemológico fundamental: la corrección sintáctica y la eficiencia algorítmica son condiciones necesarias pero no suficientes para el éxito de un sistema de software. Como documentó el CHAOS Report del Standish Group (2020), más del 40% de los proyectos fallidos tenían un código técnicamente funcional, pero no resolvían las necesidades reales de los usuarios. Este fenómeno hunde sus raíces en la crisis del software descrita por Naur y Randell (1968), donde el problema no era la incapacidad de programar, sino la incompreensión del dominio del problema.

La ingeniería de requisitos, normada por la ISO/IEC/IEEE 29148, es el antídoto contra este desperdicio. Si no se elicitán, analizan y validan correctamente las necesidades de todos los actores (incluyendo aquellos que podrían resistirse al cambio), el equipo de desarrollo construirá una solución brillante desde el punto de vista técnico pero inútil desde el operativo. Un ejemplo paradigmático es el de los sistemas de historia clínica electrónica en hospitales públicos: si no se involucra a los enfermeros de triaje en el diseño de la interfaz, estos terminarán usando papel y lápiz, ignorando el sistema por completo.

En dominios complejos, como los descritos por el modelo Cynefin (Snowden, 2002), la relación causa-efecto sólo se comprende a posteriori. Por ello, la Gestión de Productos reemplaza a la Gestión de Proyectos tradicional: en lugar de entregar un alcance cerrado, se itera con prototipos funcionales (Figma) y se validan hipótesis de valor con usuarios reales. De esta forma, el riesgo no es que el código falle en tiempo de ejecución, sino que el producto no tenga tracción en el mercado por un desalineamiento estratégico desde la fase de concepción.

Conclusiones

La resolución de los cuatro ejercicios prácticos ha permitido confirmar que el éxito de un proyecto de software no depende exclusivamente de la calidad del código, sino de la correcta gestión de requisitos, el alineamiento de las dimensiones de Pressman y la adopción de enfoques adaptativos centrados en la entrega continua de valor.

En primer lugar, la ingeniería de requisitos, normada por la ISO/IEC/IEEE 29148, se consolida como el pilar estratégico para traducir necesidades humanas en soluciones tecnológicas viables. La validación temprana mediante prototipado e inteligencia artificial reduce significativamente el costo del cambio y evita el desperdicio de recursos en funcionalidades innecesarias.

En segundo lugar, el análisis del caso de la startup de delivery evidenció que la desalineación entre Personas, Proceso, Producto y Tecnología es una causa directa del fracaso. La imposición de tecnologías complejas sin competencias reales en el equipo y la ausencia de un marco de gestión de cambios generan parálisis operativa y corrupción del alcance. El liderazgo servicial, basado en la facilitación y la protección del equipo, es la respuesta adecuada para mitigar estos riesgos.

En tercer lugar, la transición del PMBOK 6 al PMBOK 7 representa un cambio paradigmático: de la rigidez procesal a la flexibilidad basada en principios de valor. Este enfoque permite navegar la incertidumbre propia de los dominios complejos, priorizando la sostenibilidad técnica y financiera del producto. Finalmente, el mayor riesgo en el software moderno no es el error sintáctico, sino construir una solución perfecta que nadie use. La identificación temprana de todos los actores y la validación continua de hipótesis de valor son las herramientas estratégicas para garantizar que el esfuerzo técnico se traduzca en impacto real. En definitiva, la ingeniería de software es una disciplina humana antes que técnica. El equilibrio entre estándares, agilidad y liderazgo empático es la clave para entregar software que realmente transforme realidades.

Referencias

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Agile Alliance.

Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.

Greenleaf, R. K. (1977). *Servant Leadership: A Journey into the Nature of Legitimate Power and Greatness*. Paulist Press.

IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*, Version 4.0. IEEE Computer Society Press.

International Organization for Standardization. (2018). *Risk management — Guidelines* (ISO Standard No. 31000:2018). ISO.

International Organization for Standardization, & International Electrotechnical Commission. (2009). *Software and systems engineering — Software measurement — IFPUG functional size measurement method 2009* (ISO/IEC Standard No. 20926:2009). ISO/IEC. s.